Dmitry Usakov

Belgrade, Serbia   dvusakov@gmail.com   Dmitry Usakov

About me

Rust Software Engineer focused on building reliable and maintainable backend systems. I care about clear code, practical engineering decisions, and understanding how technical work supports real product needs.

I value transparent and direct communication, responsibility for my work, and calm collaboration. I prefer delivering improvements step by step, discussing trade-offs openly, and following tasks through to completion.

Work Experience

Software Engineer | *Noldor Solutions — Caffeine* *Feb 2023 — Aug 2025*

Caffeine is a geosocial networking service connecting coffee shop owners, roasteries, and coffee lovers

I owned the backend implementation of Caffeine from an early stage, building the core backend services, defining API contracts, and implementing RESTful endpoints used by the Web UI and mobile client applications. The backend was implemented with Rust, Tokio, Axum, Diesel, and PostgreSQL.

A technical highlight of this work was the polymorphic forms API for coffee shop data. I built a schema-driven backend workflow for dynamic form definitions, validation rules, submission handling, typed serialization/deserialization, and PostgreSQL persistence. This allowed different coffee shop data structures to be supported through stable API contracts without hardcoding each form variant into separate client-facing API logic.

Skills: Rust | CQRS | Microservice Architecture | Service Boundaries | Service-to-Service APIs | Service-to-Client APIs | API Contract Design | RESTful API | Tokio | Axum | Diesel | PostgreSQL | Polymorphic Forms API with Dynamic Validation | CI/CD

Side Activities

Software Engineer — Navigation Systems | *Higher League Robotics Hackathon* *2024*

Robot navigation system for a 2-day robotics hackathon

I worked as the navigation systems engineer in a 3-person team alongside a hardware engineer and a computer vision engineer. My responsibilities covered navigation logic, path tracking, robot motion behavior, technical component selection for TurtleBot3, and integration with the team's hardware and computer vision work. The navigation layer was implemented in Python 3 using ROS 2, and I iteratively tested, debugged, and tuned the robot's movement behavior under hackathon time constraints.

I designed and implemented the path-tracking system and introduced a PID controller after several testing iterations. This reduced reliance on predefined movement presets, improved trajectory stability, shortened route completion time by around 15%, and helped the team take 2nd place in the competition.

Skills: Python | ROS 2 | Robot Navigation | Motion Control | Path Tracking | Control Logic | Testing & Debugging | Iterative Prototyping | Marker-Based Navigation | PID Control | Sensor Data Processing | Hardware/Software Component Selection

Software Engineer — Maintainer | *gridstack* *Apr 2026 — Present*

Rust backend for a Matrix-compatible messaging system

I design and maintain a Matrix-compatible homeserver backend in Rust. My responsibilities cover authentication flows, room API design, non-federated Matrix Client-Server API implementation, event persistence, transactional writes, idempotent client operations, and API contract generation. The backend is implemented with Rust, Axum, Tokio, Diesel, and PostgreSQL, with the project structured around clear service boundaries and extensible room/state modeling.

I implemented a rate-limited registration and login flow with session isolation, JWT-based authentication, device binding, and conservative credential error handling. I also built the core non-federated Matrix room API surface, including authorization, room creation, membership, and /sync events. For event persistence, I worked with prev/auth event links, event depth, DAG consistency, transactional writes, and idempotent client transactions. I also introduced room-version and state-resolution boundaries and built an AST-Based OpenAPI generator that derives API contracts from Rust source code without requiring manual code annotations.

Skills: Rust | Axum | Tokio | Diesel | PostgreSQL | Matrix Client-Server API | RESTful API | Clean Architecture | Service/Repository Architecture | Domain Entities | Repository Contracts | Service Boundaries | API Boundaries | Conflict Resolution | Idempotent Transactions | Authentication Flows | Session Isolation | Device Binding | Event Persistence | Transactional Writes | Idempotency | DAG-Based Event Modeling | OpenAPI Generation | AST-Based Code Generation | Rust Source Analysis

Skills

Rust Ecosystem: Rust | Tokio | Axum | Diesel | AST-Based Code Generation
Python Ecosystem: Python | ROS 2
Software Architecture: OOP | SOLID Principles | CQRS | Clean Architecture | Service/Repository Architecture | Domain Entities | Repository Contracts | Microservice Architecture | Service Boundaries | API Boundaries | DAG-Based Event Modeling | Idempotency | Idempotent Client Transactions | Iterative Prototyping
Backend & API Design: RESTful API | Service-to-Service APIs | Service-to-Client APIs | Matrix Client-Server API | API Contract Design | OpenAPI Generation | Polymorphic Forms API with Dynamic Validation | Coffee Shop Discovery/Search
Data & Persistence: PostgreSQL | SQL | Transactional Writes | Event Persistence
Authentication & Security: Authentication & Authorization | Authentication Flows | JWT | Password Hashing
Robotics & Control Algorithms: Robot Navigation | Motion Control | Path Tracking | Control Logic | PID Control | Marker-Based Navigation | Sensor Data Processing | Iterative Prototyping | Hardware/Software Component Selection
Infrastructure & Tooling: Linux | Docker | Git | CI/CD | Testing & Debugging | Hardware/Software Component Selection

Education

RANEPA 2026
BSc in Applied Informatics in Information Security *Moscow, Russia*